Canonical source: 00-CHARTER/GUIDES/MEMORY_AND_LEARNING.md

Æ Orange AI Computer Memory And Learning

Memory exists to preserve project truth across models, accounts, restarts, and long work. Learning exists to improve future routing or execution after evidence shows what worked. Neither is permission to turn guesses into canon.

Memory Stack

Layer	Responsibility
Cold source	exact files, transcripts, artifacts, and operator records
AE Memory / Cobra	indexed facts, decisions, failures, preferences, and evidence pointers
Project Continuum	project identity, duplicate-work preflight, and resumable context
Commitment atoms	decisions and laws with continuing force
Failure Memory	prior matching failures, causes, and recovery evidence
Context Crystal	smallest source-backed workbench needed for the current task
Receipts	immutable evidence of actions and outcomes

SQLite and vector indexes are projections. They improve retrieval but never replace source artifacts, receipt chains, or explicit operator decisions.

Retrieval Contract

Retrieval combines lexical search for exact identifiers with dense search for semantic matches and hybrid ranking for both. Every useful memory result should carry project scope, source identity, time, and a path back to exact evidence.

When memories conflict, apply this order:

1. fresh semantic live probe;
2. fresh receipt for the exact runtime path;
3. current executable test;

4. current source and configuration;
5. older memory or prose.

Do not silently overwrite the losing record. Preserve the contradiction and record why the newer evidence outranks it. That history prevents a stale claim from returning after another restart.

Governed Learning

Learning begins after completed work, not before it:

```
executed result
-> verification
-> receipt
-> scoped lesson candidate
-> recurrence and counterexample checks
-> bakeoff against the incumbent
-> promote, quarantine, or reject
-> rollback path retained
```

`--learn` submits a successful order for governed learning. It does not grant a model authority to edit policy, rewrite history, promote itself, or widen tool scope. A recurring method becomes a deterministic reflex only when it beats the incumbent under a named evaluation and remains reversible.

Operator Workflow

At the start of work:

1. confirm the canonical root and target project;
2. retrieve current commitments and matching failures;
3. inspect the source pointers behind consequential memories;
4. build a sparse workset;
5. hydrate more source only when the task or audit requires it.

At the end of work:

1. verify the result through the exact runtime path;
2. emit or preserve the receipt;
3. record changed decisions, unresolved blockers, and the next entry point;
4. submit a lesson only when the evidence supports reuse;
5. record compression debt when missing context caused rework or wrong recall.

Current Bounded Evidence

The accepted held-out memory suite contains 23 retrieval cases. Hybrid retrieval passed 23/23 with MRR `0.9058`, p50 `281 ms`, and p95 `445 ms`; lexical-only passed 20/23 and dense-only passed 21/23. Three contradiction-debt cases were resolved by the evidence precedence above.

These numbers establish that benchmark's retrieval behavior. They do not prove that every project is indexed, every memory is current, or every future query will be answered correctly.

Verify

```
cd C:\AtomEons\Orange5
bun run bench:memory-quality
bun run test:learning-queue
bun run proof:learning-behavior
bun run proof:receipt-to-reflex
bun run proof:failure-memory-live
```

Read the emitted cases and receipt paths. A terminal summary without preserved inputs, expected evidence, and hashes is not an auditable memory result.

Related Guides

- Operator Manual
- Receipts and Audit
- AtomSmasher Production
- Proof and Benchmarks